
Containerization and DevOps Lab

EXPERIMENT – 04

Docker Essentials

Dockerfile, .dockerignore, Tagging and Publishing

Aim

To understand Dockerfile creation, use of .dockerignore, image tagging, multi-stage builds, and publishing Docker images to Docker Hub.

Objectives

- Containerize an application using Dockerfile
- Use .dockerignore to optimize image builds
- Build and tag Docker images
- Manage containers and images
- Implement multi-stage builds
- Publish images to Docker Hub

Part 1: Containerizing Applications with Dockerfile

Step 1: Create a Simple Application

Create Project Folder

```
mkdir my-flask-app
cd my-flask-app
```

```
C:\Users\Hp>wsl
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp$ mkdir my-flask-app
cd my-flask-app
mkdir: cannot create directory 'my-flask-app': File exists
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ docker build -t my-flask-app .          docker:default
[+] Building 0.2s (1/1) FINISHED
=> [internal] load build definition from Dockerfile          0.1s
=> => transferring dockerfile: 2B                          0.0s
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ docker build -t my-flask-app .          docker:default
[+] Building 0.1s (1/1) FINISHED
=> [internal] load build definition from Dockerfile          0.0s
=> => transferring dockerfile: 2B                          0.0s
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ mv dockerfile.txt Dockerfile
mv: cannot stat 'dockerfile.txt': No such file or directory
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ docker build -t my-flask-app .          docker:default
[+] Building 0.1s (1/1) FINISHED
=> [internal] load build definition from Dockerfile          0.0s
=> => transferring dockerfile: 2B                          0.0s
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$
```

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route('/') def hello():
    return "Hello from Docker!"

@app.route('/health') def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
Flask==2.3.3
```

```
GNU nano 7.2 requirements.txt *
Flask==2.3.3
```

Step 2: Create Dockerfile

Dockerfile

```
# Use Python base
image FROM python:3.9-
slim

# Set working
directory
WORKDIR /app

# Copy requirements
file COPY
requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application
code COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

![alt text](/Containerisation-and-devops-LAB-EXPERIMENTS/Lab/lab4/image-2.png)

Part 2: Using .dockerignore

Step 1: Create .dockerignore File

```
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ nano requirements.txt
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ ls
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ Dockerfile
app.py
requirements.txt
Dockerfile: command not found
app.py: command not found
requirements.txt: command not found
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ nano app.py
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ █
```

.dockerignore

```
# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env .ve
nv env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py
```

```
GNU nano 7.2                                .dockerignore
# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py
|
```

Step 2: Importance of .dockerignore

- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

Part 3: Building Docker Images

Step 1: Basic Build Command

```
docker build -t my-flask-app .
```

```

aryanraj@DESKTOP-DU3B1ES:~/mnt/c/Users/Hp/my-flask-app$ docker build -t my-flask-app .
docker:default

[+] Building 36.7s (11/11) FINISHED
=> [internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 204B 0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 5.9s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060e261f53f144965f755599aab1acdale13cfl731b1b 10.7s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060e261f53f144965f755599aab1acdale13cfl731b1b 0.1s
=> => sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075046c2e3df3881d 1.29MB / 1.29MB 1.0s
=> => sha256:fe74430849022d13b0d04b8969a953f842f59c6e9d1a0c2c83d710affa286c08 13.88MB / 13.88MB 6.1s
=> => sha256:ea56f685404adfb1680322f152d2cfee62115b304da481c2c450878315beb508 2518 / 2518 1.4s
=> => sha256:38513bd7256313495cd83b3b0915a633cf5475dc2a0702ab2c8d191020ca5d 29.78MB / 29.78MB 5.0s
=> => extracting sha256:38513bd7256313495cd83b3b0915a633cf5475dc2a0702ab2c8d191020ca5d 2.7s
=> => extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075046c2e3df3881d 0.4s
=> => extracting sha256:fe74430849022d13b0d04b8969a953f842f59c6e9d1a0c2c83d710affa286c08 1.9s
=> => extracting sha256:ea56f685404adfb1680322f152d2cfee62115b304da481c2c450878315beb508 0.1s
=> [internal] load build context 0.3s
=> => transferring context: 315B 0.5s
=> [2/5] WORKDIR /app 0.1s
=> [3/5] COPY requirements.txt . 0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt 14.9s
=> [5/5] COPY app.py . 0.2s
=> => exporting to image 3.5s
=> => exporting layers 2.1s
=> => exporting manifest sha256:330c04d920b6b20807e0ed91521f949ab748b1fe4a197e4058a2b04999e621af 0.0s
=> => exporting config sha256:fd8635acfed4dada33c0989c09e59d33225c2 0.0s
=> => exporting attestation manifest sha256:5eb5a57c553d9d7256732a22a47c883570733fec79c148894f847f44dbb116 0.1s
=> => exporting manifest list sha256:c912148ba63cb46a8a847e64670a0c58201380390f6f86a1366b47cee8bfe89d 0.0s
=> => naming to docker.io/library/my-flask-app:latest 0.0s
=> => unpacking to docker.io/library/my-flask-app:latest 1.0s
aryanraj@DESKTOP-DU3B1ES:~/mnt/c/Users/Hp/my-flask-app$

```

Check built images:

docker images

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-flask-app	latest	f3e32346adc8	48 seconds ago	132MB
flask-app	latest	cb172ca9dcba	6 days ago	129MB
my-flask-app	v1	110d8f28256c	6 days ago	134MB
ubuntu	latest	493218ed0f40	5 weeks ago	78.1MB
python	3.9-slim	085da638e1b8	3 months ago	122MB

Step 2: Tagging Images

Tag with version: `docker build -t`

`my-flask-app:1.0 .`

```

failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  5.12kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> f09372aac9cf
Step 3/7 : COPY requirements.txt .
--> Using cache
--> 3b9a2e18c32c
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> ef9c85f3710f
Step 5/7 : COPY app.py .
--> Using cache
--> 4ec34f187ea9
Step 6/7 : EXPOSE 5000
--> Using cache
--> c379a290cbe5
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> f3e32346adc8
Successfully built f3e32346adc8
Successfully tagged my-flask-app:1.0

```

Multiple tags:

`docker build -t my-flask-app:latest -t my-flask-app:1.0 .`

```

failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  5.12kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> f09372aac9cf
Step 3/7 : COPY requirements.txt .
--> Using cache
--> 3b9a2e18c32c
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> ef9c85f3710f
Step 5/7 : COPY app.py .
--> Using cache
--> 4ec34f187ea9
Step 6/7 : EXPOSE 5000
--> Using cache
--> c379a290cbe5
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> f3e32346adc8
Successfully built f3e32346adc8
Successfully tagged my-flask-app:latest
Successfully tagged my-flask-app:1.0

```

Tag for Docker Hub: `docker build -t`

`username/my-flask-app:1.0 .`

```

failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  5.12kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> f09372aac9cf
Step 3/7 : COPY requirements.txt .
--> Using cache
--> 3b9a2e18c32c
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> ef9c85f3710f
Step 5/7 : COPY app.py .
--> Using cache
--> 4ec34f187ea9
Step 6/7 : EXPOSE 5000
--> Using cache
--> c379a290cbe5
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> f3e32346adc8
Successfully built f3e32346adc8
Successfully tagged username/my-flask-app:1.0

```

Tag existing image:

```
docker tag my-flask-app:latest my-flask-app:v1.0
```

Step 3: View Image Details

List images:

```
docker images
```

Image

history:

```
docker history my-flask-
```

app Inspect image: docker

```
inspect my-flask-app
```

```
aryanraj@nt/d/1b4/my-flask-app$ docker tag my-flask-app:latest my-flask-app:v1.0
aryanraj@nt/d/1b4/my-flask-app$ docker images
REPOSITORY          TAG                 CREATED            SIZE
my-flask-app         latest             3 minutes ago     132MB
my-flask-app         v1.0               3 minutes ago     132MB
username/my-flask-app latest             6 days ago        132MB
flask-app            latest             6 days ago        133MB
ubuntu              latest             3 months ago      78.1MB
0146d794a29ba       0146dc9a          3 months ago      78.1MB
aryanraj/testimage  3.9-tilm hba     about a year ago  122MB
aryanraj@nt/d/1b4/my-flask-app$ docker history my-flask-app
IMAGE ID   CREATED      CREATED BY          SIZE    COMMENT
f3e32346adc3 3 minutes   /bin/sh -c #(nop) CMD ["python" "app.py"] 0B
cf79c993d9c3 3 minutes   /bin/sh -c #(nop) EXPOSE 5000              0B
8a596f1789f 3 minutes   /bin/sh -c #(nop) COPY file:494a3994ac532_228 buildkit.dockerfile.v0
047058737fbc 3 minutes   /bin/sh -c pip install, file-cache-dir -- 10.5MB
75683346eb64 3 minutes   /bin/sh -c #(nop) WORKDIR                  339B
0146d79aa    3 months ago (latest)                0B    buildkit.dockerfile.v0
064c770c3908 3 months ago /bin/sh n3"l=command"                0B    buildkit.dockerfile.v0
065a863a1be0 4 months ago + debian sh -arch "angate-nit - pi/.. 78.0MB buildkit.dockerfile.v0

aryanraj@nt/d/1b4/my-flask-app$ docker history my-flask-app
IMAGE ID   CREATED      CREATED BY          SIZE    COMMENT
f3e32346adc3 3 minutes   /bin/sh -c #(nop) CMD ["python" "a_ 0B
cf79c993d9c3 3 minutes   /bin/sh -c #(nop) EXPOSE 5000              0B
8a596f1789f 3 minutes   /bin/sh -c #(nop) COPY file:494a3994_228 buildkit.dockerfile.v0
047058737fbc 3 minutes   /bin/sh -c pip install, file-cache__ 1059B buildkit.dockerfile.v0
75683346eb64 3 minutes   /bin/sh -c #(nop) WORKDIR                  339B buildkit.dockerfile.v0
0146d79aa    3 minutes ago (latest)                0B    buildkit.dockerfile.v0
064c770c3908 3 months ago /bin/sh n3"l=command"                0B    buildkit.dockerfile.v0
065a863a1be0 3 months ago + debian sh -arch "amasa-nt/p_ 01MB buildkit.dockerfile.v0
cdccing> 4 months ago debuerretype          78.6MB debuerretype 0.16

aryanraj@nt/d/1b4/my-flask-app$ docker tag my-flask-app:latest my-flask-app:v1.0
```



```

aryanraj@mnt:/mnt/My-Flask-App$ docker inspect my-flask-app
[
  {
    "ID": "sha256:f3a32346d8b43c3b0a4b0d7aef9b0c27a9b01b8f41d73233aef1",
    "RepoTags": [
      "my-flask-app:1.0",
      "my-flask-app:latest",
      "my-flask-app:v1.0",
      "myname/my-flask-app:1.0"
    ],
    "RepoDigests": [],
    "Parent": "sha256:c77b29b0c6a7f6a077033c7b6b05a01af3a080b01d4a0e0a18f09b0a00",
    "Comment": "",
    "Created": "2020-02-17T16:18:19.83886032Z",
    "ContainerVersion": "38.2.3",
    "Author": "",
    "Config": {
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Env": [
        "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=4D965ADCECFC60312370D0483A0EB312D70D9B4",
        "PYTHON_VERSION=3.6.5",
        "PYTHON_SHA256=8d8c1b053956162e13515e6c1d60c0b948953639477721c8996c8134"
      ],
      "Cmd": [
        "python",
        "app.py"
      ],
      "WorkingDir": "/app"
    },
    "Architecture": "amd64",
    "Os": "linux",
    "Size": 13488607,
    "GraphDriver": {
      "Data": {
        "MergedDir": "/var/lib/docker/overlay2/.../merged",
        "UpperDir": "/var/lib/docker/overlay2/.../diff",
        "WorkDir": "/var/lib/docker/overlay2/.../work"
      },
      "Name": "overlay2"
    },
    "RootFS": {
      "Type": "layers",
      "Layers": [ "sha256:...", "sha256:..." ]
    },
    "Metadata": {
      "LastTagTime": "2020-02-17T22:15:35.098832Z"
    }
  }
]

```

Part 4: Running and Managing Containers

Step 1: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-
```

app Test application: `curl http://localhost:5000` View

running containers: `docker ps` View logs:

```
docker logs flask-container
```

```

aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ docker run -p 5000:5000 my-flask-a
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a pr
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.4:5000
* Press CTRL+C to quit
172.17.0.1 - - [17/Feb/2026 22:10:11] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [17/Feb/2026 22:10:12] "GET /favicon.ico HTTP/1.1" 404 -

```



Step 2: Manage Containers

Stop container: `docker stop`

`flask-container` **Start**

container: `docker start`

`flask-container` **Remove**

container: `docker rm`

`flask-container` **Force**

remove:

`docker rm -f flask-container`

Explanation

`docker rm` removes a stopped container.

- Docker does not allow removal of a running container.
docker rm -f forcefully stops and removes it.
If the container is already deleted, Docker

returns: Error response from daemon: No such

container This is expected behavior.

Part 5: Multi-Stage Builds

Why Multi-stage Builds?

- Smaller final image size
- Better security
- Separate build and runtime environments

Multi-stage Dockerfile

Docker file.multistage

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

COPY requirements.txt .

RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY app.py .

RUN useradd -m -u 1000 appuser
USER appuser
```

EXPOSE 5000

CMD ["python", "app.py"]

```
GNU nano 7.2 Dockerfile.multistage
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

Build and Compare

Build regular image:

```
docker build -t flask-regular .
```

```
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ docker build -t flask-regular
ERROR: docker: 'docker buildx build' requires 1 argument

Usage:  docker buildx build [OPTIONS] PATH | URL | -

Run 'docker buildx build --help' for more information
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$
```

Build multi-stage image:

```
docker build -f Dockerfile.multistage -t flask-multistage .
```

```
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ docker build -f Dockerfile.multistage -t flask-multistage .
[+] Building 0.3s (1/1) FINISHED
=> [internal] load build definition from Dockerfile.multistage    0.1s
=> => transferring dockerfile: 2B                                0.0s
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile.multistage: no such file or directory
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$
```

Compare sizes:

docker images | [grep](#) flask-

Expected

- flask-regular → flask-multistage →

```
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$ docker build -f Dockerfile.multistage
[+] Building 0.3s (1/1) FINISHED
=> [internal] load build definition from Dockerfile.multistage    0.1s
=> => transferring dockerfile: 28                                0.0s
failed to build: failed to solve: failed to read dockerfile: open Dockerfile.multistage: no s
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp/my-flask-app$
```

Part 6: Publishing to Docker Hub

Step 1: Login and Tag

docker login

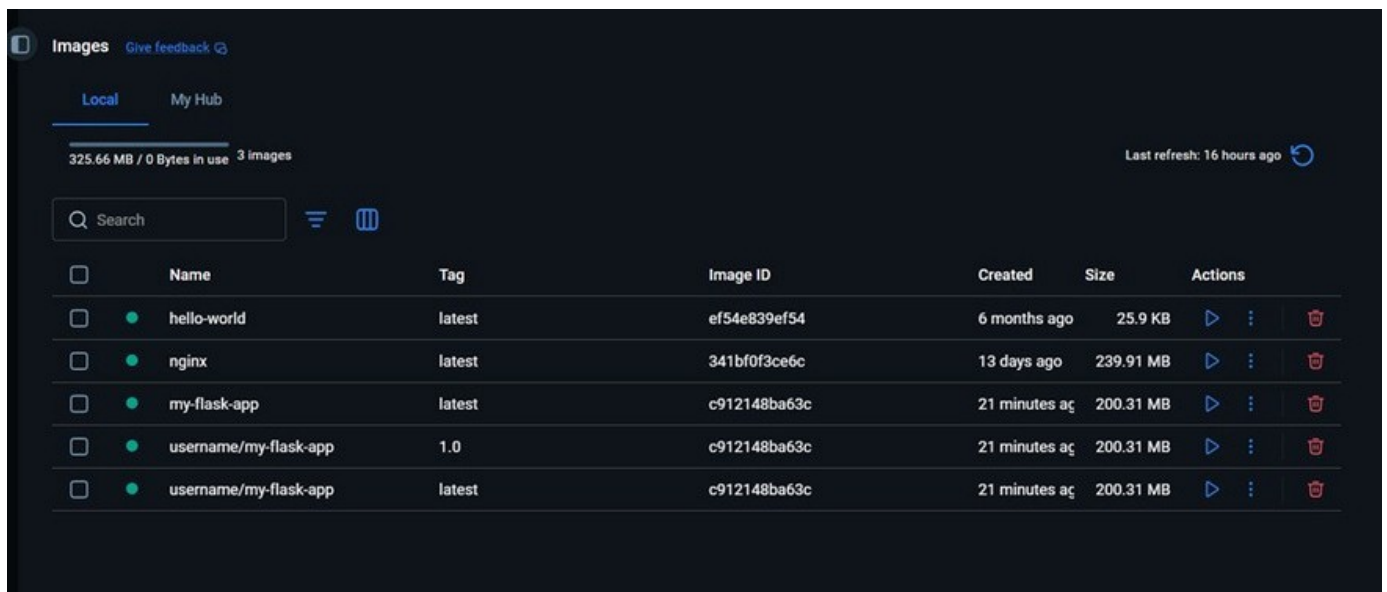
Tag image:

```
docker tag my-flask-app:latest username/my-flask-app:1.0
docker tag my-flask-app:latest username/my-flask-
app:latest
```

Push image:

```
docker push username/my-flask-app:1.0
docker push username/my-flask-
app:latest
```

```
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp$ docker tag my-flask-app:latest username/my-flask-app:1.0
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp$ docker tag my-flask-app:latest username/my-flask-app:latest
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp$ docker push username/my-flask-app:1.0
The push refers to repository [docker.io/username/my-flask-app]
16b987c7d7ca: Waiting
fc7443b849b2: Waiting
7de724d8022f: Waiting
ea56f685484a: Waiting
b3ec39b36ae8: Waiting
b25d456a8a9c: Waiting
cbaf350992c1: Waiting
ef9a77890e7d: Waiting
38513bd72563: Waiting
push access denied, repository does not exist or may require authorization: server message: insufficient_
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp$ docker push username/my-flask-app:latest
The push refers to repository [docker.io/username/my-flask-app]
7de724d8022f: Waiting
ef9a77890e7d: Waiting
ea56f685484a: Waiting
16b987c7d7ca: Waiting
fc7443b849b2: Waiting
cbaf350992c1: Waiting
b25d456a8a9c: Waiting
38513bd72563: Waiting
b3ec39b36ae8: Waiting
push access denied, repository does not exist or may require authorization: server message: insufficient_
aryanraj@DESKTOP-DU3B1ES:/mnt/c/Users/Hp$
```



Step 2: Pull and Run from Docker Hub

On another machine:

```
docker pull username/my-flask-app:latest
```

Run the pulled image:

```
docker run -d -p 5000:5000 username/my-flask-app:latest
```

Result

The Flask application was successfully containerized using Dockerfile. Image tagging, multi-stage builds, and publishing to Docker Hub were performed successfully.

Conclusion